บทที่ 2

โพรเซส

2.1 บทนำ

ในการทำงานของระบบปฏิบัติการ โพรเซส (Process) ถือเป็นหนึ่งในองค์ประกอบพื้นฐานที่สำคัญที่สุด ซึ่งหมายถึงโปรแกรมที่กำลังทำงานอยู่ภายในหน่วยความจำหลัก โดยโพรเซสแต่ละตัวจะได้รับการจัดสรรทรัพยากรของระบบ เช่น หน่วยประมวลผลกลาง หน่วยความจำ และอุปกรณ์รับส่งข้อมูล เพื่อดำเนินงานตามหน้าที่ของตน โพรเซสสามารถเปลี่ยนสถานะได้หลายรูปแบบระหว่างการทำงาน ตั้งแต่เริ่มต้น จนถึงจบกระบวนการ และในการจัดการโพรเซสนี้ ระบบปฏิบัติการจะใช้กลไกต่าง ๆ เช่น การจัดตารางโพรเซส (process scheduling), การสลับบริบท (context switch), การจัดเก็บข้อมูลในบล็อกควบคุมกระบวนการ (process control block: PCB) รวมถึงการสร้างและยุติโพรเซสอย่างมีลำดับขั้น โพรเซสแต่ละตัวอาจมีโพรเซสย่อย หรือเรียกว่า “โพรเซสลูก” ที่ทำงานร่วมกับ “โพรเซสแม่” ตามลำดับชั้นของโครงสร้างที่ระบบปฏิบัติการควบคุมอย่างต่อเนื่อง

นอกจากนี้ ในระบบที่มีการทำงานพร้อมกันหลายโปรเซส (multiprogramming หรือ multitasking) การติดต่อสื่อสารระหว่างโพรเซส (inter-process communication: IPC) จึงกลายเป็นสิ่งสำคัญ โดยโพรเซสสามารถสื่อสารกันผ่านหน่วยความจำร่วม (shared memory) หรือการส่งข้อความ (message passing) ซึ่งต้องอาศัยกลไกการประสานงาน เช่น การใช้บัฟเฟอร์ การซิงโครไนซ์ และการควบคุมลำดับในการรับส่งข้อมูล เพื่อป้องกันปัญหาการสูญหายของข้อมูล การซ้อนทับ หรือการตายของโพรเซสโดยไม่ตั้งใจ ระบบปฏิบัติการจึงจำเป็นต้องออกแบบโครงสร้างและกลไกอย่างเหมาะสม เพื่อรับประกันการทำงานของโพรเซสให้มีประสิทธิภาพ ถูกต้อง และปลอดภัย ซึ่งจะเป็นหัวใจสำคัญในการเข้าใจการทำงานโดยรวมของระบบคอมพิวเตอร์ในระดับล่าง

2.2 แนวคิดของโพรเซส

ในยุคเริ่มต้นของระบบคอมพิวเตอร์ การประมวลผลข้อมูลเป็นไปในรูปแบบของ**โมโนโปรแกรมมิ่ง** (mono programming) ซึ่งหมายถึงระบบสามารถประมวลผลได้เพียงหนึ่งงานในแต่ละช่วงเวลา ระบบลักษณะนี้มีข้อจำกัดสำคัญ คือการใช้ทรัพยากรของระบบอย่างไม่มีประสิทธิภาพ เนื่องจากมีช่วงเวลาที่ทรัพยากร เช่น หน่วยประมวลผลกลาง (CPU) ว่างเปล่าโดยไม่สามารถนำมาใช้ประโยชน์ได้

ต่อมา ได้มีการพัฒนาระบบให้รองรับ**การทำงานหลายงานพร้อมกัน** (multiprogramming) โดยการแบ่งโปรแกรมออกเป็นส่วนย่อยที่สามารถทำงานได้อย่างอิสระ ส่วนย่อยเหล่านี้เรียกว่า**โพรเซส** (Process)

2.2.1 ความหมายและโครงสร้างของโพรเซส

โพรเซส (Process) หมายถึง โปรแกรมที่กำลังถูกประมวลผลอยู่ในหน่วยความจำหลัก โดยโพรเซสหนึ่ง ๆ ประกอบด้วยองค์ประกอบหลายส่วน ได้แก่

- คำสั่งของโปรแกรม (Program Instructions)
- ค่าของ `Program Counter` ซึ่งระบุตำแหน่งคำสั่งถัดไปที่จะประมวลผล
- ค่าของรีจิสเตอร์ที่จำเป็นต่อการทำงาน
- สถานะการทำงานของโพรเซส เช่น `Running` `Ready` หรือ `Waiting`

โพรเซสอาจกล่าวได้ว่าคือ กลุ่มของหน่วยความจำ (memory cells) ที่มีการเปลี่ยนแปลงตามลำดับของคำสั่งในโปรแกรม โดยมีหน่วยประมวลผลกลางทำหน้าที่ตีความคำสั่งและควบคุมการทำงาน โพรเซสต้องอาศัยทรัพยากรของระบบ เช่น หน่วยประมวลผลกลาง หน่วยความจำ และอุปกรณ์รับส่งข้อมูล (I/O Devices) เพื่อให้ทำงานได้อย่างมีประสิทธิภาพ

2.2.2 หน้าที่ของระบบปฏิบัติการในการจัดการโพรเซส

ระบบปฏิบัติการมีบทบาทสำคัญในการควบคุมและจัดการโพรเซส โดยมีหน้าที่หลักดังนี้:

1. **การสร้างโพรเซส** (Process Creation): โหลดโปรแกรมเข้าสู่หน่วยความจำและเตรียมพร้อมให้ทำงาน
2. **การจัดตารางการใช้หน่วยประมวลผลกลาง** (CPU Scheduling): กำหนดลำดับการใช้หน่วยประมวลผลกลางของแต่ละโพรเซส
3. **การทำลายโพรเซส** (Process Termination): ยุติการทำงานของโพรเซสเมื่อเสร็จสิ้น
4. **การจัดสรรทรัพยากร** (Resource Allocation): จัดสรรหน่วยประมวลผลกลาง หน่วยความจำ และอุปกรณ์ I/O ให้โพรเซส

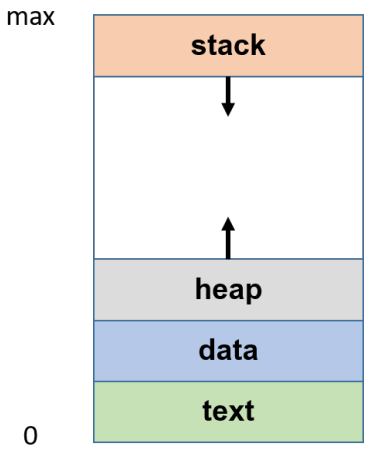
ระบบปฏิบัติการต้องบริหารจัดการทรัพยากรที่มีอยู่อย่างจำกัด เพื่อหลีกเลี่ยงปัญหาความขัดแย้งของการใช้งานทรัพยากร และเพื่อให้แต่ละโพรเซสสามารถดำเนินการได้อย่างราบรื่น

2.2.3 โครงสร้างของโพรเซสในหน่วยความจำ

เมื่อเปิดใช้งานโปรแกรม ระบบจะโหลดโปรแกรมเข้าสู่หน่วยความจำ และเปลี่ยนสถานะเป็นโพรเซส ในขั้นตอนนี้ โพรเซสจะใช้พื้นที่หน่วยความจำหลักที่แบ่งออกเป็นสองส่วนสำคัญ ได้แก่:

1. **Stack**: ใช้จัดเก็บค่าพารามิเตอร์ ตัวแปรภายใน และข้อมูลสำหรับการเรียกใช้ฟังก์ชัน
2. **Heap**: ใช้สำหรับการจัดเก็บข้อมูลแบบไดนามิก โดยสามารถร้องขอและคืนพื้นที่ในระหว่างการทำงาน

รูปที่ 2.1 แสดงให้เห็นถึงการจัดสรรหน่วยความจำของโพรเซส ซึ่งประกอบด้วยส่วนคำสั่ง (Text), ส่วนข้อมูล (Data), Stack และ Heap อย่างเป็นระเบียบ



ภาพที่ 2.1: โพรเซสในหน่วยความจำหลัก

2.2.4 การทำงานของโพรเซสในสภาพแวดล้อมที่แตกต่างกัน

แม้ว่าโพรเซสจะถูกสร้างจากโปรแกรมเดียวกัน แต่ในการทำงานจริงภายใต้ระบบคอมพิวเตอร์ โพรเซสแต่ละตัวอาจมีพฤติกรรมที่แตกต่างกันได้ ตัวอย่างเช่น โพรเซสอาจถูกแยกออกเป็นหลายโพรเซสย่อยเพื่อทำงานในลักษณะสลับกัน (Interleaving Execution) หรือทำงานแบบขนาน (Concurrent Execution) โดยขึ้นอยู่กับกลไกของระบบปฏิบัติการและนโยบายการจัดสรรทรัพยากร

ในบางกรณี โพรเซสอาจไม่ทำงานโดยตรงบนฮาร์ดแวร์จริง แต่ทำงานภายใต้**สภาพแวดล้อมจำลอง** (Virtualized Environment) เช่น โปรแกรมที่พัฒนาด้วยภาษา Java เมื่อถูกคอมไพล์เป็นไฟล์นามสกุล `.class` แล้ว จะไม่ทำงานโดยตรงกับระบบปฏิบัติการ แต่จะถูกรันบน **Java Virtual Machine (JVM)** ซึ่งทำหน้าที่แปลคำสั่งเป็นภาษาที่เครื่องคอมพิวเตอร์สามารถเข้าใจและประมวลผลได้ต่อไป โพรเซสในลักษณะนี้จึงขึ้นอยู่กับซอฟต์แวร์กลางที่จัดการการแปลภาษาคำสั่งและควบคุมการทำงาน

2.2.5 คุณสมบัติของโพรเซส

เพื่อให้สามารถบริหารจัดการโพรเซสได้อย่างมีประสิทธิภาพ ระบบปฏิบัติการจะกำหนดคุณสมบัติเฉพาะของแต่ละโพรเซส เช่น ระดับความสำคัญ (Priority) และขอบเขตการทำงาน (Scope of Operation) ดังรายละเอียดต่อไปนี้:

ระดับความสำคัญ (Priority):

- ทุกโพรเซสจะได้รับการกำหนดระดับความสำคัญในขณะที่ถูกสร้างขึ้น
- บางระบบปฏิบัติการอนุญาตให้ผู้ใช้หรือโปรแกรมปรับเปลี่ยนระดับความสำคัญของโพรเซสได้ ในขณะที่บางระบบไม่อนุญาต
- โพรเซสที่มีระดับความสำคัญสูงกว่าจะได้รับสิทธิพิเศษ เช่น โอกาสในการเข้าถึงหน่วยประมวลผลกลางก่อน หรือได้รับเวลาประมวลผลมากกว่าโพรเซสอื่น ๆ

ขอบเขตการทำงาน (Scope of Operation):

- โพรเซสแต่ละตัวมีขอบเขตที่จำกัดว่าอนุญาตให้ทำอะไรได้บ้างภายในระบบ

- ขอบเขตนี้รวมถึงการเข้าถึงอุปกรณ์หรือทรัพยากร เช่น ดิสก์ หน่วยความจำ หรือการสื่อสารระหว่างโพรเซส (IPC)
- ตัวอย่างเช่น โพรเซส A อาจไม่สามารถเข้าถึงดิสก์ใด ๆ ได้ แต่สามารถส่งและรับข้อมูลกับโพรเซสอื่น ๆ ภายในระบบได้อย่างอิสระ

คุณสมบัติพิเศษอื่น ๆ:

- ระบบปฏิบัติการอาจกำหนดคุณสมบัติเพิ่มเติมให้กับโพรเซส เช่น ความสามารถในการทำงานแบบ Background, การรองรับการขัดจังหวะ (Interrupt Handling), หรือการจัดลำดับแบบ Real-Time
- คุณสมบัติเหล่านี้ถูกกำหนดขึ้นเพื่อให้โพรเซสเหมาะสมกับลักษณะงานเฉพาะและเพื่อให้การบริหารจัดการของระบบมีประสิทธิภาพสูงสุด

2.3 สถานะของโพรเซส

ขณะที่โพรเซสแต่ละตัวกำลังทำงาน สถานะของโพรเซส (process states) จะมีการเปลี่ยนแปลงตามลำดับการประมวลผล โดยระบบปฏิบัติการจะควบคุมสถานะเหล่านี้ เพื่อให้การจัดการโพรเซสเป็นไปอย่างมีประสิทธิภาพ สถานะหลักของโพรเซสมีทั้งหมด 5 สถานะ ดังแสดงในรูปที่ 2.2 และมีรายละเอียดดังนี้:

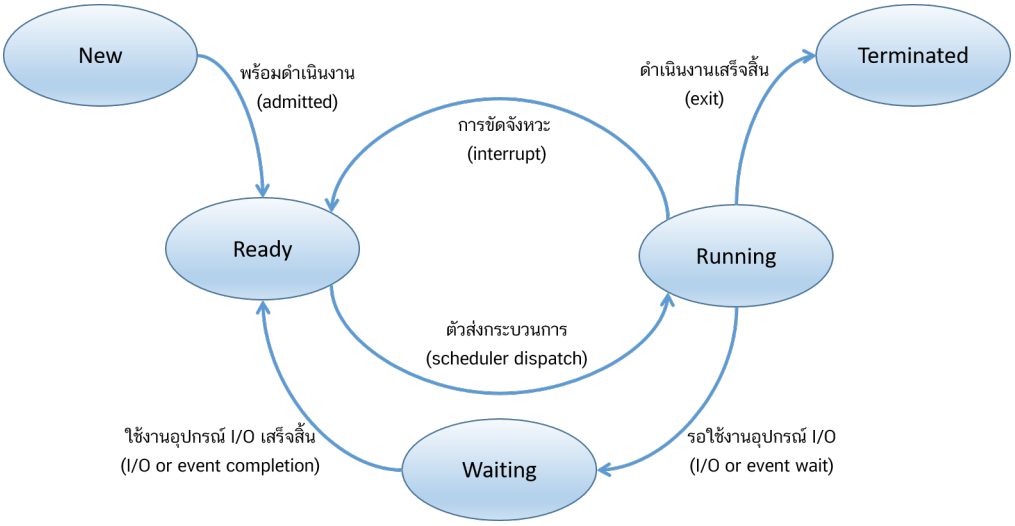
1. **สถานะสร้าง (New)**: โพรเซสอยู่ระหว่างการสร้างและกำลังจองพื้นที่ในหน่วยความจำ
2. **สถานะพร้อม (Ready)**: โพรเซสพร้อมสำหรับการประมวลผล แต่ยังไม่ได้รับการจัดสรรหน่วยประมวลผลกลาง
3. **สถานะกำลังทำงาน (Running)**: โพรเซสได้รับหน่วยประมวลผลกลาง และกำลังถูกประมวลผล
4. **สถานะรอ (Waiting)**: โพรเซสรอเหตุการณ์ภายนอก เช่น การอ่านข้อมูลจากอุปกรณ์ I/O
5. **สถานะสิ้นสุด (Terminated)**: โพรเซสทำงานเสร็จสิ้นและพร้อมถูกลบออกจากระบบ

2.4 บล็อกควบคุมกระบวนการ (Process Control Block: PCB)

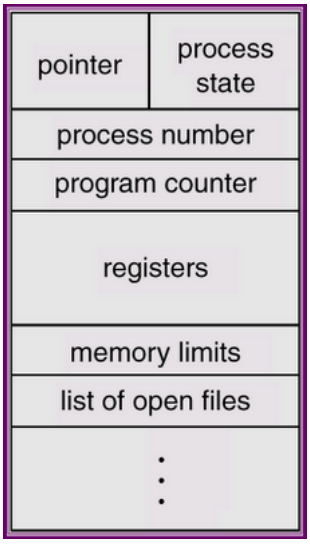
เนื่องจากระบบอาจมีหลายโพรเซสทำงานสลับกัน ระบบปฏิบัติการจึงต้องเก็บข้อมูลรายละเอียดของแต่ละโพรเซสไว้ในโครงสร้างข้อมูลที่เรียกว่า **บล็อกควบคุมกระบวนการ (Process Control Block: PCB)** ซึ่งเป็นข้อมูลสำคัญสำหรับการควบคุมและติดตามสถานะของโพรเซสนั้น ๆ ดังแสดงในรูปที่ 2.3

ข้อมูลที่เก็บใน PCB ได้แก่:

1. **Pointer**: ชี้ไปยังที่อยู่ของโพรเซสในหน่วยความจำ
2. **Process State**: แสดงสถานะปัจจุบันของโพรเซส (New, Ready, Running, Waiting, Terminated)
3. **Process ID (PID)**: หมายเลขประจำตัวของโพรเซส ซึ่งต้องไม่ซ้ำกัน
4. **Program Counter**: ตำแหน่งคำสั่งถัดไปที่โพรเซสจะประมวลผล



ภาพที่ 2.2: สถานะของกระบวนการ



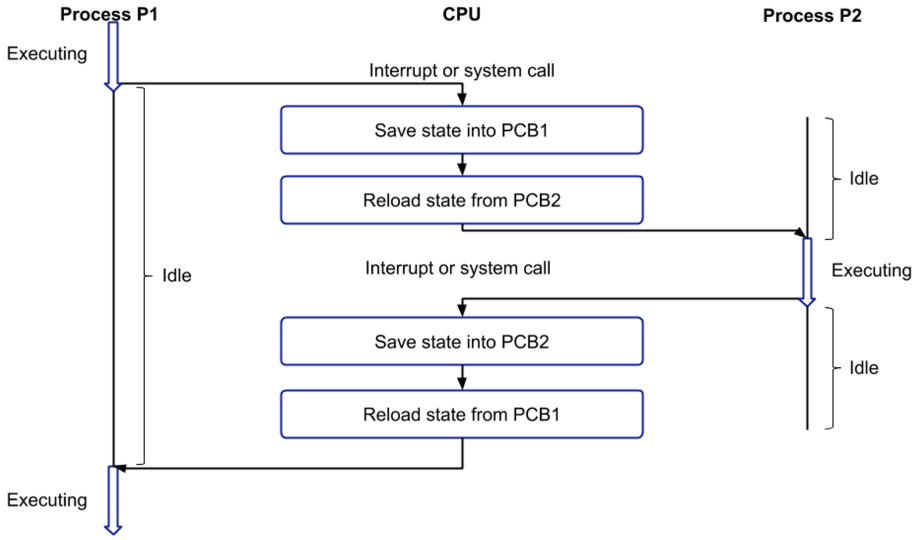
ภาพที่ 2.3: องค์ประกอบของ PCB

5. **CPU Registers**: ค่าต่าง ๆ ในรีจิสเตอร์ขณะเกิดการสลับโพรเซส (Context Switch)
6. **CPU Scheduling Information**: ข้อมูลการจัดลำดับ เช่น ค่าความสำคัญ (Priority)
7. **Memory Management Information**: ข้อมูลที่เกี่ยวกับการจัดการหน่วยความจำ เช่น Page Table และ Segment Table
8. **Accounting Information**: ข้อมูลทางบัญชี เช่น เวลาการใช้งานหน่วยประมวลผลกลาง หมายเลขบัญชีงาน
9. **I/O Status Information**: รายการอุปกรณ์ I/O ที่ใช้งาน และไฟล์ที่เปิดใช้อยู่

PCB ทำหน้าที่คล้ายแฟ้มประวัติของโพรเซสแต่ละตัว โดยระบบปฏิบัติการจะอ้างอิงข้อมูลใน PCB เพื่อควบคุมการทำงานและดำเนินการสลับโพรเซสอย่างมีประสิทธิภาพ

2.5 การสลับโพรเซส (Process Switching)

เมื่อระบบปฏิบัติการเกิดการ**ขัดจังหวะ** (Interrupt) หรือมีการเปลี่ยนลำดับการใช้หน่วยประมวลผลกลางจากโพรเซสหนึ่งไปยังอีกโพรเซสหนึ่ง จำเป็นต้องบันทึกสถานะของโพรเซสเดิมไว้ใน PCB และเรียกคืนสถานะของโพรเซสใหม่จาก PCB ของมันเอง ดังแสดงในภาพที่ 2.4



ภาพที่ 2.4: การสลับการทำงานระหว่างโพรเซส P1 และ P2

ลำดับการทำงานมีดังนี้:

1. โพรเซส P1 ทำงานอยู่ แล้วเกิดการขัดจังหวะ (Interrupt)
2. ระบบปฏิบัติการบันทึกสถานะของ P1 ลงใน PCB1
3. จากนั้น สลับการทำงานไปยังโพรเซส P2 โดยโหลดสถานะจาก PCB2
4. เมื่อต้องสลับกลับ ระบบจะใช้ข้อมูลใน PCB เพื่อเรียกคืนสถานะเดิมของแต่ละโพรเซส

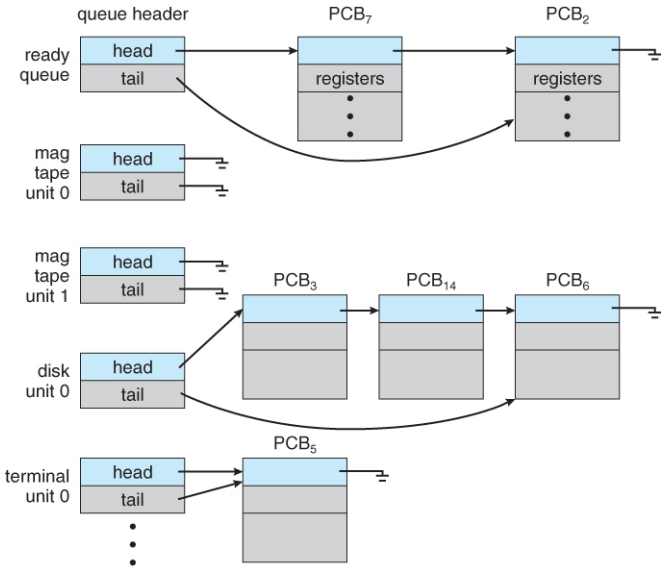
กลไกนี้ช่วยให้โพรเซสหลายตัวสามารถทำงานร่วมกันได้อย่างต่อเนื่อง แม้จะมีการแย่งใช้หน่วยประมวลผลกลางเกิดขึ้นอย่างสม่ำเสมอ

2.6 การจัดตารางของโพรเซส (Process Scheduling)

ในระบบคอมพิวเตอร์ยุคเริ่มแรกที่ใช้แนวคิดแบบ **โมโนโปรแกรมมิ่ง** (Monoprogramming) จะมีเพียงหนึ่งโพรเซสเท่านั้นที่สามารถใช้หน่วยประมวลผลกลาง (CPU) ได้ในแต่ละช่วงเวลา ซึ่งทำให้การใช้ทรัพยากรไม่คุ้มค่า ต่อมา ระบบได้พัฒนาเป็น **มัลติโปรแกรมมิ่ง** (Multiprogramming) ซึ่งสามารถจัดการหลายโพรเซสในเวลาเดียวกัน แม้ว่าระบบจะมีหน่วยประมวลผลกลางเพียงหนึ่งตัวก็ตาม โพรเซสต่าง ๆ จะสลับกันใช้งานหน่วยประมวลผลกลางโดยการควบคุมของระบบปฏิบัติการ เพื่อให้หน่วยประมวลผลกลางไม่ว่างงาน และเกิดประสิทธิภาพในการทำงานสูงสุด

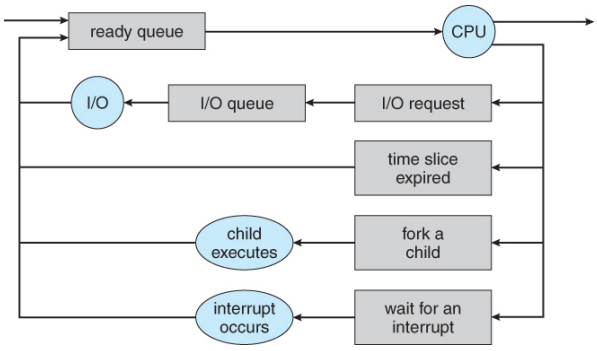
2.6.1 การจัดตารางแถวคอย (Scheduling Queues)

เมื่อโพรเซสเข้าสู่ระบบ จะเริ่มต้นที่ **Job Queue** ซึ่งเก็บไว้ในหน่วยเก็บข้อมูลขนาดใหญ่ หลังจากนั้นเมื่อโพรเซสถูกโหลดเข้าสู่หน่วยความจำหลัก จะถูกจัดเก็บใน **Ready Queue** ซึ่งเป็นรายการโพรเซสที่พร้อมรอการใช้หน่วยประมวลผลกลาง โดยโพรเซสในแถวนี้จะรอจนกว่า จะได้รับการจัดสรรเวลาหน่วยประมวลผลกลางตามลำดับ



ภาพที่ 2.5: โครงสร้างแถวคอยของโพรเซสในระบบ

หากโพรเซสกำลังทำงานและต้องรอเหตุการณ์บางอย่าง เช่น การใช้อุปกรณ์ I/O หรือการสร้างโพรเซสย่อย โพรเซสนั้นจะถูกย้ายไปยัง **Device Queue** เพื่อรอให้เงื่อนไขดังกล่าวสำเร็จ แล้วจึงย้ายกลับไปยัง Ready Queue เพื่อรอใช้งานหน่วยประมวลผลกลางอีกครั้ง ดังแสดงในแผนภาพ 2.6



ภาพที่ 2.6: ภาพแสดงการเคลื่อนย้ายโพรเซสระหว่างคิวต่าง ๆ

สถานการณ์ที่ทำให้โพรเซสหยุดชั่วคราวและออกจากหน่วยประมวลผลกลาง:

1. โพรเซสร้องขอใช้งานอุปกรณ์ I/O
2. โพรเซสสร้างโพรเซสย่อยและรอผลลัพธ์
3. โพรเซสถูกขัดจังหวะโดยระบบ

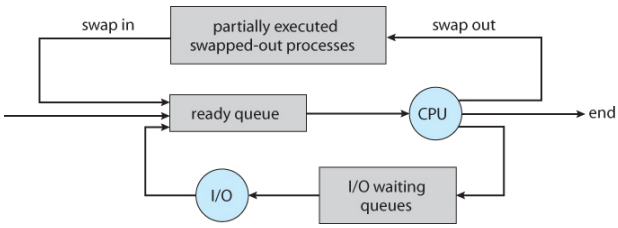
2.6.2 ตัวกำหนดการ (Schedulers)

ระบบปฏิบัติการใช้โปรแกรมพิเศษเรียกว่า **Scheduler** เพื่อเลือกและจัดลำดับโพรเซสในระบบ ซึ่งมีอยู่ 3 ประเภท ได้แก่:

1. **ตัวกำหนดการระยะยาว (Long-term Scheduler)** หรือ *Job Scheduler*: ทำหน้าที่เลือกโพรเซสจาก Job Queue เพื่อเข้าสู่หน่วยความจำหลักและ Ready Queue

2. **ตัวกำหนดการระยะสั้น (Short-term Scheduler)** หรือ *CPU Scheduler*: เลือกโพรเซสจาก Ready Queue เพื่อมอบหมายหน่วยประมวลผลกลางให้ประมวลผล

3. **ตัวกำหนดการระยะกลาง (Medium-term Scheduler)**: ใช้ในระบบที่มีการสลับโพรเซสออกจากหน่วยความจำหลักเพื่อลดภาระ และโหลดกลับเข้ามาใหม่เมื่อพร้อมทำงาน (ดังแสดงในรูปที่ 2.7)



ภาพที่ 2.7: หน้าที่ของตัวกำหนดการระยะกลาง

ความแตกต่างของ Scheduler แต่ละประเภท:

- **Long-term Scheduler** ทำงานไม่บ่อย เน้นควบคุมสมดุลระหว่างโพรเซสที่เน้น I/O กับโพรเซสที่เน้นหน่วยประมวลผลกลาง

- **Short-term Scheduler** ทำงานถี่มาก (ทุก 2-10 มิลลิวินาที) เพื่อรองรับการเปลี่ยนแปลงสถานะโพรเซสอย่างต่อเนื่อง

- **Medium-term Scheduler** ใช้สำหรับการ *Swapping* โพรเซสเข้า-ออกหน่วยความจำหลัก

2.6.3 ระบบมัลติทาสกิ้ง (Multitasking) บนมือถือ

ระบบปฏิบัติการยุคใหม่ เช่น iOS และ Android มีลักษณะการจัดการโพรเซสที่แตกต่างกันในเชิงนโยบาย:

- **iOS**: อนุญาตให้มีเพียงหนึ่งโพรเซสที่ทำงานบนหน้าจอ (Foreground Process) ขณะที่โพรเซสอื่น ๆ จะอยู่เบื้องหลังและถูกจำกัดการทำงาน

- **Android**: อนุญาตให้โพรเซสเบื้องหลังทำงานต่อไปได้ เช่น บริการระบบ หรือการดาวน์โหลดที่ไม่ได้แสดงผลกับผู้ใช้โดยตรง

2.6.4 การสลับการทำงานของหน่วยประมวลผล (Context Switch)

Context Switch หมายถึง กระบวนการที่ระบบปฏิบัติการหยุดโพรเซสที่กำลังทำงานอยู่ และสลับไปยังโพรเซสอื่น โดยจะต้อง:

1. บันทึกสถานะของโพรเซสปัจจุบัน (เช่น ค่า Register, Program Counter) ลงใน **PCB**
2. โหลดสถานะของโพรเซสใหม่จาก **PCB** แล้วดำเนินการต่อ

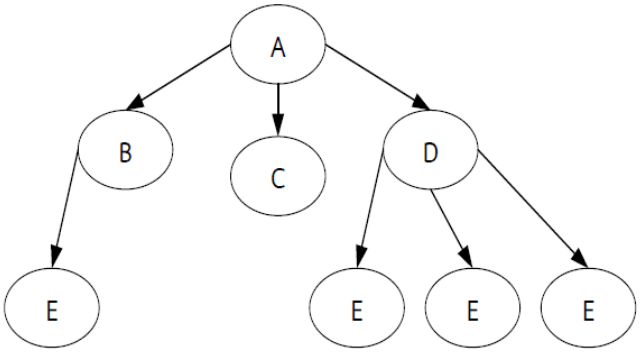
แม้ว่าการสลับนี้จะใช้เวลาสั้น ๆ แต่ก็ส่งผลต่อประสิทธิภาพโดยรวม โดยเฉพาะหาก Context Switch เกิดขึ้นบ่อยเกินไป ซึ่งในอนาคต ระบบปฏิบัติการจะใช้แนวคิด **เธรด (Threads)** เพื่อช่วยลดภาระในการสลับ Context ในระดับโพรเซส

2.7 การดำเนินการของโพรเซส

2.7.1 แนวคิดการดำเนินการพื้นฐานของโพรเซส

ในระบบปฏิบัติการสมัยใหม่ มีโพรเซสจำนวนมากที่สามารถทำงานพร้อมกันได้ โดยระบบปฏิบัติการจะจัดการโพรเซสเหล่านี้ผ่านกลไกการสร้าง การดำเนินการ และการสิ้นสุดของโพรเซส โพรเซสสามารถแบ่งเป็นโพรเซสแม่ และโพรเซสลูก ซึ่งโพรเซสแม่มีสิทธิ์สร้างและควบคุมโพรเซสลูกได้

1. เมื่อผู้ใช้ร้องขอให้ทำงาน ระบบจะสร้างโพรเซสใหม่ขึ้นมาสำหรับงานนั้น
2. ระบบปฏิบัติการเองก็มีโพรเซสเฉพาะของระบบเพื่อควบคุมการทำงาน
3. โพรเซสสามารถสร้างโพรเซสย่อยเพิ่มเติมเพื่อแบ่งเบาภาระงาน
4. โดยทั่วไปเมื่อโพรเซสแม่สิ้นสุด โพรเซสลูกทั้งหมดจะถูกยุติด้วย (Cascading termination)



ภาพที่ 2.8: ลำดับชั้นของโพรเซส

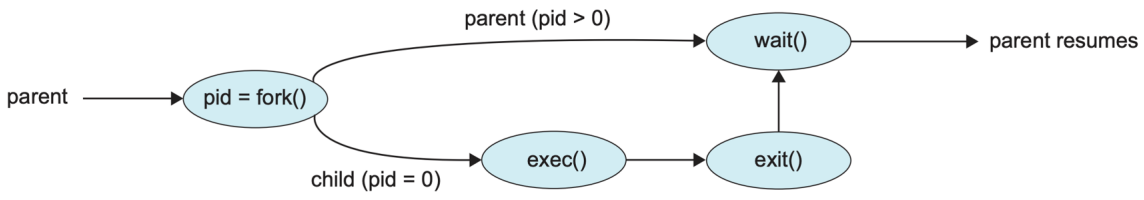
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
 pid_t pid;

  /* fork a child process */
 pid = fork();

 if (pid < 0) { /* child process */
  fprintf(stderr, "Fork Failed");
  return 1;
}
  else if (pid ==0) { /*child process */
   execlp ("/bin/ls", "ls, NULL);
}
else {  /* parent process */
 /* parent will wait for the child to complete */
wait (NULL);
printf("Child Complete");
}
   return 0;
}
```

ภาพที่ 2.9: ตัวอย่างโค้ดภาษาซีในการสร้างโพรเซส



ภาพที่ 2.10: การสร้างโพรเซสด้วยฟังก์ชัน `fork()`

2.7.2 การสร้างโพรเซส (Process Creation)

การสร้างโพรเซสใหม่ในระบบปฏิบัติการ Unix ใช้ system call ชื่อว่า `fork()` ซึ่งจะสร้างโพรเซสใหม่ที่เหมือนกับโพรเซสแม่ โดยสามารถใช้ร่วมกับ `exec()` เพื่อเรียกใช้โพรเซสใหม่ และใช้ `wait()` เพื่อรอให้โพรเซสลูกสิ้นสุดการทำงาน

2.7.3 การสิ้นสุดของโพรเซส (Process Termination)

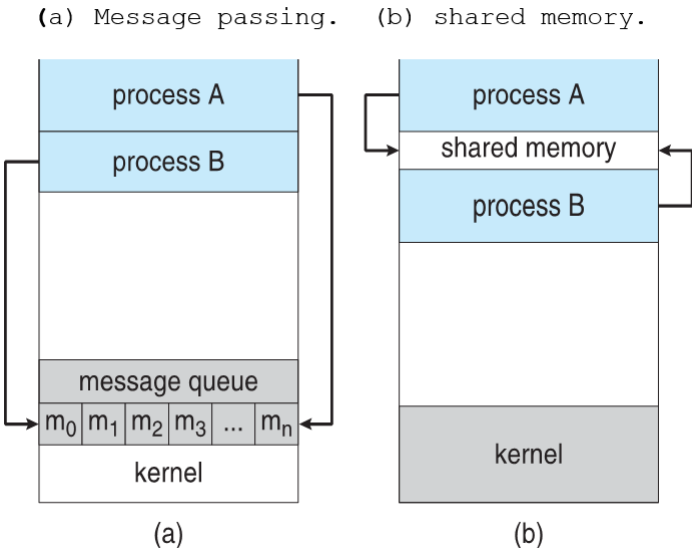
โพรเซสจะสิ้นสุดเมื่อคำสั่งสุดท้ายถูกประมวลผล หรือระบบเรียกใช้ system call `exit()` เพื่อลบโพรเซสออกจากระบบ หากโพรเซสแม่สิ้นสุด โพรเซสลูกทั้งหมดจะถูกยุติด้วย แต่ในบางกรณี โพรเซสแม่สามารถยุติโพรเซสลูกได้ด้วย system call `abort()`

2.8 การสื่อสารระหว่างโพรเซส

โพรเซสในระบบสามารถแบ่งออกเป็น 2 ประเภท คือ:

- **โพรเซสอิสระ (Independent)**: ไม่ติดต่อหรือใช้ทรัพยากรร่วมกับโพรเซสอื่น
- **โพรเซสร่วมมือ (Cooperative)**: แบ่งปันข้อมูลและต้องประสานการทำงานร่วมกัน

ความจำเป็นในการสื่อสารระหว่างโพรเซส (Inter-Process Communication: IPC) เช่น การแบ่งปันข้อมูล (Shared files) การประมวลผลแบบขนาน (Speedup) การออกแบบแบบแยกโมดูล (Modularity) และความสะดวกของผู้ใช้ (Convenience)



ภาพที่ 2.11: รูปแบบการสื่อสารระหว่างโพรเซส

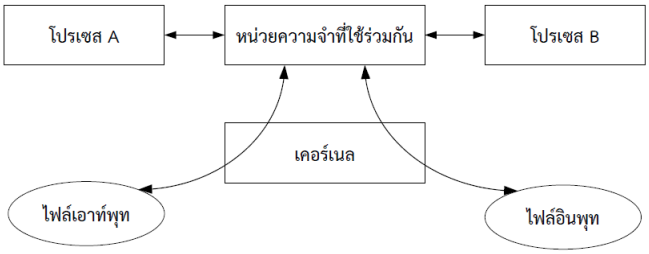
2.8.1 รูปแบบของการสื่อสารระหว่างโพรเซส

การสื่อสารระหว่างโพรเซส (IPC) แบ่งออกเป็น 2 กลุ่มใหญ่:

Shared Memory

โพรเซสสามารถใช้หน่วยความจำร่วมกันผ่านตำแหน่งที่กำหนดไว้ ซึ่งต้องใช้การประสานงาน (Synchronization) เพื่อไม่ให้ข้อมูลผิดพลาด เช่น ปัญหา Producer-Consumer เป็นต้น

ตัวอย่างบัฟเฟอร์ที่ใช้ร่วมกันได้แก่: บัฟเฟอร์ไม่จำกัด (Unbounded) และบัฟเฟอร์จำกัด (Bounded)



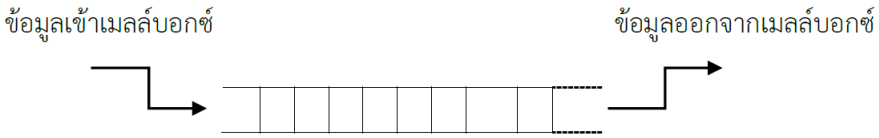
ภาพที่ 2.12: การส่งข้อมูลผ่าน Shared Memory

Message Passing

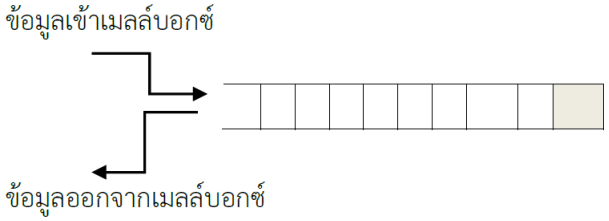
การสื่อสารผ่านข้อความระหว่างโพรเซสสามารถทำได้ทั้งแบบ: การสื่อสารทางตรง (Direct Communication) การสื่อสารทางอ้อมผ่าน Mailbox การตั้งชื่อแบบ Symmetric/Asymmetric และการสื่อสารแบบ Blocking/ Non-blocking



ภาพที่ 2.13: กล่องข้อความแบบคิว FIFO



ภาพที่ 2.14: กล่องข้อความแบบไปป์ (Pipe)



ภาพที่ 2.15: กล่องข้อความแบบสแต็ก (Stack)

ผู้พัฒนาระบบต้องเลือกกลไกการสื่อสารระหว่างโพรเซส ให้เหมาะสมกับบริบทของแอปพลิเคชันและจำนวนโพรเซสที่ต้องประสานงาน เพื่อให้ระบบมีเสถียรภาพ และประสิทธิภาพสูงสุด

2.9 การใช้บัฟเฟอร์ในการส่งข้อความระหว่างโพรเซส

การใช้บัฟเฟอร์ในการสื่อสารระหว่างโพรเซส ไม่ได้เป็นเพียงการกำหนดเส้นทางข้อมูลเท่านั้น แต่ยังรวมถึงความสามารถในการเก็บข้อความชั่วคราวระหว่างการส่งและรับ ซึ่งระบบสามารถกำหนดขนาดความจุของบัฟเฟอร์ได้ตามลักษณะการทำงาน โดยแบ่งเป็น 3 ประเภทหลัก ดังนี้:

1. **ความจุค่าศูนย์ (Zero Capacity):** หมายถึง บัฟเฟอร์ที่ไม่มีพื้นที่เก็บข้อมูลเลย ผู้ส่งต้องรอจนกว่าผู้รับจะรับข้อมูลก่อนจึงจะสามารถส่งข้อความต่อได้ เรียกอีกอย่างว่าระบบไม่มีบัฟเฟอร์ (No Buffering System)

2. **ความจุค่าจำกัด (Bounded Capacity):** บัฟเฟอร์สามารถเก็บข้อมูลได้สูงสุด n รายการ หากบัฟเฟอร์ยังไม่เต็ม ผู้ส่งสามารถส่งข้อมูลได้โดยไม่ต้องรอ แต่หากเต็มแล้ว ผู้ส่งต้องรอจนกว่าจะมีพื้นที่ว่าง

3. **ความจุค่าไม่จำกัด (Unbounded Capacity):** ไม่มีขีดจำกัดในการเก็บข้อมูล ผู้ส่งสามารถส่งข้อมูลได้เรื่อย ๆ โดยไม่ต้องรอ

ในกรณีที่ใช้บัฟเฟอร์ ผู้ส่งอาจไม่ทราบว่าข้อความที่ส่งไปถึงผู้รับแล้วหรือไม่ หากข้อมูลมีความสำคัญมาก ระบบจะต้องมีวิธีในการยืนยันผลการรับส่ง เช่น:

- `send(Q, message);`
- `receive(Q, message);`
- `receive(P, message);`
- `send(P, "acknowledgment");`

สถานการณ์นี้เรียกว่าแบบ **Synchronous Communication** ซึ่งผู้ส่งจะรอการยืนยันจากผู้รับก่อนดำเนินการต่อ อย่างไรก็ตาม ยังมีรูปแบบพิเศษอีกสองแบบคือ:

1. **การส่งโดยไม่รอการตอบรับ:** แต่หากผู้รับยังไม่พร้อม ข้อมูลอาจสูญหายได้ เหมาะกับข้อมูลที่ไม่ต้องการความถูกต้องสูง

2. **การตอบกลับแบบหน่วงเวลา (Delayed Reply):** ผู้ส่งจะถูกบล็อกจนกว่าจะได้รับการตอบกลับจากผู้รับ ข้อความและการตอบกลับจะมีขนาดคงที่ เช่น 8 คำ (words)

2.10 เงื่อนไขควรคำนึงในการสื่อสารระหว่างโพรเซส

การส่งข้อความในระบบคอมพิวเตอร์ โดยเฉพาะในระบบแบบกระจาย (Distributed System) อาจเกิดข้อผิดพลาดขึ้นได้ ซึ่งต้องมีการออกแบบกลไกเพื่อรับมือกับสถานการณ์เหล่านี้ ตัวอย่างข้อยกเว้นที่พบได้บ่อยได้แก่:

2.10.1 การสิ้นสุดของโพรเซสที่เกี่ยวข้อง

หากโพรเซสผู้ส่งหรือผู้รับสิ้นสุดก่อนที่การสื่อสารจะเสร็จสมบูรณ์ จะส่งผลให้เกิดความล้มเหลวในการส่งข้อความ ดังนี้:

1. **ผู้รับสิ้นสุดก่อน:** ผู้ส่งจะรอการตอบกลับตลอดไป หากไม่มีการจัดการ อาจทำให้ผู้ส่งถูกบล็อกตลอดเวลา
2. **ผู้ส่งสิ้นสุดก่อน:** ข้อมูลอาจไม่ถูกส่งถึงผู้รับ หรือผู้รับอาจรอข้อความที่ไม่มีวันมาถึง

แนวทางแก้ไขอาจรวมถึงการตรวจสอบสถานะของโพรเซสโดยระบบปฏิบัติการ และแจ้งโพรเซสที่เกี่ยวข้องเมื่อโพรเซสอื่นสิ้นสุดลง

2.10.2 การสูญหายของข้อความระหว่างทาง

ปัญหาการสูญหายของข้อความอาจเกิดจากข้อผิดพลาดด้านฮาร์ดแวร์หรือเครือข่าย การแก้ปัญหาสามารถทำได้โดยใช้วิธีใดวิธีหนึ่งต่อไปนี้:

1. ให้ระบบปฏิบัติการตรวจสอบการส่ง และทำการส่งใหม่โดยอัตโนมัติหากตรวจพบการสูญหาย
2. ให้โพรเซสผู้ส่งตรวจสอบการส่งเอง โดยอาจใช้กลไกเช่น acknowledgment หรือ checksum
3. ระบบปฏิบัติการตรวจสอบ และแจ้งโพรเซสผู้ส่งให้ดำเนินการส่งซ้ำ

กลไกที่นิยมใช้คือการตั้งเวลา **Timeout** ซึ่งหากไม่มีสัญญาณตอบรับภายในเวลาที่กำหนด จะถือว่าข้อความสูญหาย และระบบจะทำการส่งใหม่โดยอัตโนมัติ หรือแจ้งโพรเซสให้ดำเนินการแก้ไขต่อไป